

Antro

Feiras Agroecológicas do Recife

Projeto de EDOO • CIn UFPE

Equipe 7: João Bernardo Gomes Alves de Carvalho, Lucas Medrado Santos, Maria Luiza de Paula Portela e Luis Eduardo Lustosa Brito

Código-fonte: <https://github.com/joaobgdev/Antro>

1. Configurações do Sistema

O programa foi desenvolvido com o framework Qt. O frontend foi construído com Qt Quick e QML, e o componente StackView gerencia a navegação entre as telas. O backend foi implementado em C++17, seguindo os princípios da programação orientada a objetos (POO). A compilação é configurada com CMake.

O armazenamento e o gerenciamento dos dados foram feitos com SQLite, por meio do módulo Qt SQL: contas de usuários (login e cadastro), feiras, produtos e reservas. Os dados são locais: o arquivo `antro.db` é criado na pasta de dados do aplicativo e não precisa ser incluído no repositório.

2. Proposta e Funcionamento

O Antro é um aplicativo que aproxima pequenos produtores agroecológicos dos compradores que frequentam as feiras do Recife. Os produtores divulgam seus produtos na plataforma, e os compradores interessados selecionam e reservam os itens para retirar na feira, no dia e horário escolhidos.

O sistema possui dois perfis: **Comprador** e **Feirante**. As feiras são previamente cadastradas no banco de dados. O feirante escolhe em quais feiras participa e cadastra seus produtos, que passam a ser oferecidos em todas as feiras selecionadas, e responde aos pedidos recebidos. O comprador começa pela lista de feiras, escolhe uma, vê os produtores que participam dela, seleciona produtos, monta o carrinho (sacola) e solicita a reserva.

2.1 Reservas

A plataforma organiza a solicitação e oferece horários disponíveis para que o comprador escolha quando deseja buscar o produto. O feirante pode aceitar ou recusar o pedido; o comprador acompanha a situação ou cancela a reserva. O valor mostrado é uma

estimativa, pois produtos vendidos por peso podem variar na pesagem. **O pagamento é combinado diretamente entre comprador e produtor, fora do aplicativo.**

2.2 Produtos

Para oferecer produtos, o feirante primeiro seleciona as feiras em que participa (tela *Alterar perfil*) e depois informa, para cada mercadoria, o nome, a unidade de venda (unidade, kg ou 100 g), o preço e a quantidade disponível em estoque. Ao salvar, os produtos ficam disponíveis para os compradores reservarem. O estoque de cada produto é único e compartilhado entre todas as feiras em que ele é oferecido.

2.3 Login / Cadastro

O login é feito com telefone e senha, e os dados ficam no SQLite local. Para criar uma conta como comprador é necessário informar nome completo, telefone e senha. Para o cadastro como feirante, além disso, são exigidos o nome da banca e uma identificação OCS (Organização de Controle Social), mecanismo que permite a organização de agricultores familiares para garantir a qualidade dos produtos vendidos diretamente ao consumidor, especialmente no contexto da produção orgânica.

O sistema faz validações básicas: nome com pelo menos duas palavras; telefone com 10 ou 11 dígitos (DDD + número), único no sistema; senha com no mínimo 6 caracteres; e, para o feirante, banca e código OCS preenchidos. As senhas **não** são guardadas em texto puro: o banco guarda o hash SHA-256 da senha com um sal aleatório, repetido 10.000 vezes. Devido às limitações de tempo e à complexidade de integração com serviços externos, o sistema não consegue garantir a autenticidade dos dados informados nem a legitimidade do código OCS. Em versões futuras do projeto, poderia haver um mecanismo de administração para verificação manual dos cadastros de feirantes e a integração com APIs externas para automatizar essa validação.

2.4 Feiras e agenda de retirada

As quatro feiras abaixo são inseridas no banco na primeira execução. Dia e horário são dados cadastrados; mudanças de funcionamento precisam ser confirmadas com a organização de cada feira.

Feira	Local	Funcionamento
Feira Agroecológica da Várzea	Praça da Várzea	Sábados, 7h às 10h
Espaço Agroecológico das Graças	Rua Andrade de Souza	Sábados, 4h às 11h

Feira	Local	Funcionamento
Feira de Produtos Orgânicos de Casa Forte	Praça da Vitória Régia	Sábados, 5h às 11h
Espaço Agroecológico do Sítio da Trindade	Sítio da Trindade (Casa Amarela)	Sábados, 5h às 11h

A classe `AgendaFeira` calcula, no fuso America/Recife, quais feiras estão em andamento (elas aparecem primeiro na tela inicial), até cinco datas de retirada nos próximos 35 dias e janelas de até uma hora, sem as que já terminaram. A escolha é validada de novo na hora de gravar a reserva.

3. Arquitetura e organização

O projeto está dividido em telas, controladores, classes de domínio e repositórios: as **telas QML** mostram informações e recebem as ações do usuário; os **controladores** (`AuthController`, `CompradorController` e `VendedorController`) validam essas ações e organizam o fluxo; os **repositórios** (`RepositorioUsuario` e `RepositorioCatalogo`) executam as consultas e alterações no banco `antro.db`. Os arquivos se organizam em `main.cpp` e `Main.qml` (inicialização e janela com `StackView`, que começa no login), `telas/` (páginas QML), `include/` e `src/` (classes, divididas em `models`, `services` e `ui`), `assets/`, `tests/`, `docs/` (página de apresentação em HTML) e `CMakeLists.txt` (C++17, Qt Quick, Qt SQL e o executável `appAntro`).

3.1 Responsabilidade das classes principais

`AuthController` mantém a sessão (cadastro, login e saída). `CompradorController` cuida de feiras, produtores, carrinho, agendamento e reservas; `VendedorController`, da participação nas feiras, dos produtos e dos pedidos. Ambos recebem uma referência à autenticação para conferir quem usa o sistema. `RepositorioUsuario` guarda as contas, `RepositorioCatalogo` guarda feiras, produtos e reservas, e `CatalogoComprador` mantém o catálogo e o carrinho em memória.

4. Classes e orientação a objetos

Uma classe define dados e operações; um objeto é uma instância dessa classe em execução. No Antro, um objeto `Produto` reúne nome, preço e estoque, enquanto objetos `Agricultor` e `Consumidor` representam os perfis de usuário.

4.1 Encapsulamento e herança

`Produto` mantém seus atributos privados e oferece métodos como `getPreco()`, `getEstoque()`, `deduzirEstoque()` e `reporEstoque()`; por exemplo, `deduzirEstoque()` recusa quantidades inválidas, maiores que o estoque ou fora do passo de venda. `Usuario` possui nome e telefone protegidos, compartilhados pelas classes derivadas. `Agricultor` acrescenta, como atributos privados, o nome da banca e o código OCS; `Consumidor` reutiliza os dados comuns.

4.2 Classe abstrata e polimorfismo

`Usuario` é abstrata porque `getPerfil()` e `getSubtitulo()` são métodos virtuais puros (`= 0`); por isso o programa cria um `Agricultor` ou um `Consumidor`, nunca um `Usuario` genérico. O `override` informa que a classe derivada implementa um método virtual da base. `AuthController` guarda o usuário logado em `std::unique_ptr<Usuario>`: ao chamar `getPerfil()`, o resultado depende do objeto armazenado ("comprador" ou "feirante"). O mesmo vale para `validar()`, em que o agricultor reaproveita a validação comum e acrescenta as exigências de banca e OCS.

4.3 Ponteiros, referências e composição

O `unique_ptr` libera o usuário ao trocar de conta ou encerrar a sessão.

`QPointer<AuthController>` permite que os outros controladores acompanhem a autenticação sem serem donos dela. Parâmetros `const &`, como `const QString &nome`, evitam cópias. Os controladores possuem seus repositórios como atributos, formando uma relação de composição. `Pedido`, `ItemPedido` e `GerenciadorFeira` fazem parte do modelo inicial do projeto e não são usados pelo fluxo atual da interface, que trabalha com `CatalogoComprador` e `RepositorioCatalogo`.

5. Qt e o início do programa

O Qt mantém a lógica em C++ e a interface em QML, e oferece recursos de janelas, listas, eventos e banco de dados; o Qt Creator é o ambiente usado para editar, compilar e executar o projeto. Muitas classes do Qt começam com Q, como `QString` (texto), `QObject` (comunicação entre objetos), `QSqlDatabase` (conexão com o banco) e `QSqlQuery` (consultas).

Em `main.cpp`, `QGuiApplication` prepara o aplicativo gráfico, `QQmlApplicationEngine` interpreta o QML, `loadFromModule("Antro", "Main")` carrega a tela principal e `exec()` mantém o programa tratando cliques, teclado e outros eventos. Os controladores usam

`Q_OBJECT` (metaobjetos), `Q_PROPERTY` (propriedades lidas pelo QML, com `NOTIFY` avisando mudanças), `Q_INVOKABLE` (métodos C++ chamados pelas telas, como `entrar()` ou `finalizarReserva()`) e `QML_SINGLETON` (uma instância compartilhada pelas telas).

Os sinais mantêm tudo sincronizado: `catalogoChanged` (vendedor) chama `recarregar()` no controlador do comprador, e `reservasChanged` (comprador) chama `atualizarPedidos()` no do vendedor.

6. Estruturas de dados utilizadas

Estrutura	Onde é usada	Para quê
<code>std::vector</code> / <code>QVector</code>	<code>CatalogoComprador</code> (feiras, produtos, ofertas, carrinho)	Coleções em ordem; o carrinho é um vetor e cada item é identificado pela posição. Os elementos são localizados por busca linear (<code>buscarProduto()</code> , <code>temOferta()</code>), adequada ao pequeno volume de dados.
<code>std::map</code> e <code>std::queue</code>	<code>GerenciadorFeira</code> (modelo inicial)	Catálogo indexado pelo id do produto e fila FIFO de pedidos.
<code>QMap<QString, int></code>	<code>salvarReservas()</code>	Agrupar os itens do carrinho por "vendedor:feira", uma reserva por par.
<code>std::stable_sort</code>	<code>CompradorController::feiras()</code>	Ordenar as feiras: em andamento, data mais próxima e horário de início.

7. Produtos, reservas e banco de dados

O catálogo do comprador usa os mesmos registros mantidos pelo agricultor. As tabelas `participacoes` e `ofertas` ligam produtores e produtos às feiras; o banco também tem `users`, `feiras`, `vendedores`, `produtos`, `reservas` e `reserva_itens`, e uma migração única converte dados de versões anteriores. No CRUD, `adicionarProduto()` e

`salvarPerfil()` criam; `carregarPerfil()` e `carregar()` leem; `definirPreco()` e `definirEstoque()` atualizam; `removerProduto()` exclui, marcando o registro como removido para preservar o histórico das reservas.

Preço, unidade e quantidade. Os produtos podem ser vendidos por **unidade** (quantidade inteira), **kg** (passos de 0,5 kg) ou **100 g** (passos de 0,1 kg). O agricultor informa o preço de 100 g, mas o banco guarda o preço por kg para manter os cálculos coerentes.

Do carrinho à retirada. O carrinho só aceita quantidades dentro do passo de venda e do estoque. O comprador escolhe data e janela para cada feira, e `finalizarReserva()` envia os itens ao repositório, que cria uma reserva por produtor e feira. Antes de gravar, são conferidos perfil, oferta, quantidade, preço, unidade, estoque e horário. A gravação usa uma transação: se um item falha, tudo é desfeito. Ao solicitar, o estoque é reduzido; recusa ou cancelamento o devolvem uma única vez.

Situação	Significado
SOLICITADA	Pedido enviado, aguardando a resposta do agricultor.
ACEITA	O agricultor aceitou (não é possível depois que o horário terminou).
RECUSADA / CANCELADA	Recusada pelo agricultor ou cancelada pelo comprador; o estoque é devolvido.
RETIRADA	O agricultor registrou a entrega, a partir do início do horário agendado.

8. Participação da equipe

Integrante	Contribuições
João Bernardo	Criação do projeto; cadastro, login, SQL e integração das classes; ajustes no carrinho e nas telas de entrada.
Lucas Medrado Santos	Área do comprador; padronização visual; integração de catálogo, estoque, agendamento e reservas.
Maria Luiza Portela	Organização dos arquivos; modelos de produto e pedido; telas do agricultor; integração na <code>main</code> .

Integrante	Contribuições
Luis Eduardo	Unificação de produtos e feiras; agendamento e acompanhamento de reservas.

9. Cronologia do desenvolvimento

Data	O que foi feito
21/08	Definição de três membros; discussão de ideias com visão computacional.
31/08	Entrada do quarto membro (Luis); brainstorming por WhatsApp.
01/09	Criação do repositório no GitHub.
03/09	Brainstorming no Discord; possível tema: remix musical com visão computacional (a estudar).
08/09	Criação de um Docs para organizar o passo a passo.
14/09	Criação de uma página no Trello para organizar os próximos passos.
15/09	Reunião presencial sobre o andamento.
22/09	Escolha definitiva do tema (aplicativo que conecta pequenos produtores agroecológicos a compradores); visão computacional descartada pelo prazo.
23/09	Separação das tarefas de programação de cada membro.
25/09	Versão lo-fi do projeto; ideação do nome.
26/09	Discussões sobre telas e funcionamento geral; início da identidade visual; breve reunião.
27/09	Início do desenvolvimento, com as telas de login e cadastro.
29/09	Criação das pastas <code>src</code> , <code>include</code> e <code>ui</code> e dos arquivos com os nomes dos modelos. Pendências: erro na área de produto (dependia de consumidor) e falta de <code>Usuario</code> como classe base.

Data	O que foi feito
30/09	Adição de <code>Usuario</code> , <code>Consumidor</code> e <code>Agricultor</code> ; correção de bugs para executar o aplicativo.
01/10	Reunião presencial para alinhar a visão do sistema.
03/10	Telas da área do comprador: feiras, vendedores, produtos e carrinho de compras.
04/10	Padronização do visual das telas do comprador.
05/10	Telas da área do vendedor: relatório de compras, perfil, alteração de perfil, feira, produtos e valores.